

Optimization Algorithms (OptAlgos)

Programming Assignment – Winter Semester 2025/26

Version 01

Programming Language:

Ideally, you should implement the assignment on your own laptop, which you will also bring to the examination appointment. Java, C, C++, C#, Python, Scala, and Rust are acceptable (Python is discouraged); for other programming languages please ask beforehand, though it should not be a problem.

Algorithms:

Implement the following algorithms from the lecture:

- Local Search
- Greedy

The local search should start from deliberately poor initial solutions so the algorithms can achieve convincing improvements.

Generic Implementation:

Important: The requirement described below—to implement the algorithms generically—is crucial for your program to be accepted!

The implementations must be as generic as presented in the lecture. This means the implementations must not contain any information about the optimization problem, the neighborhood definition (local search), or the selection strategy (greedy). Your algorithm implementation must not need to be changed in order to apply it to an entirely different, unforeseen optimization problem or to the same optimization problem with another neighborhood/selection strategy. Likewise, the implementation of the optimization problem must not contain information specific to the algorithms or the neighborhood/selection strategy.

In object-oriented languages, this usually means polymorphism: all details of the optimization problem and the neighborhood/selection strategy are hidden behind interfaces, and implementing classes for the optimization problem contain no knowledge of the neighborhood/selection strategies.

Generics/templates are also possible (e.g., in C++). In Java they may be less suitable, but the decision is yours.

Algorithmic Problem:

You apply your algorithms to the following concrete optimization problem:

Given a finite set of rectangles with integer side lengths and a box side length L . You may assume no rectangle side is longer than L .

Find an axis-parallel placement of all rectangles in the plane such that any two rectangles are openly disjoint: they may share vertices or boundary segments but no interior points. Rectangles may be rotated by 90° . Each rectangle must lie completely inside a square of side length L , referred to as a box. Minimize the number of boxes required to place all rectangles.

Instance Generation:

Your instance generator takes as input L , the number of rectangles, and a lower and upper bound for both side lengths. It generates the required number of rectangles uniformly at random within the given bounds, independently.

Neighborhoods for Local Search:

Implement the following neighborhoods:

1. Geometry-based:

A neighbor can be created by directly moving rectangles, within a box or between boxes. You have freedom in how to do this. You may modify the objective function to reward steps that move the solution closer to improvement, even if the true objective does not yet improve—for example, rewarding steps that reduce the number of rectangles in already-sparse boxes.

2. Rule-based:

Instead of valid placements, local search operates on permutations of rectangles. Like greedy, rectangles are placed in the order of the permutation. The neighborhood consists of small modifications to the permutation. You may consider relocating rectangles from sparse boxes to other positions in the permutation.

3. Allowing partial overlaps:

Modify the geometry-based neighborhood so that rectangles may overlap up to a specified percentage. The overlap is intersection area divided by the larger rectangle's area. Begin at 100% allowed overlap (trivial optimal solution), then gradually reduce. Penalize violations strongly in the objective. Ultimately produce an overlap-free solution.

Selection Strategies for Greedy:

Implement two fundamentally different selection strategies determining the order in which rectangles are selected and how each is placed disjointly relative to previously placed rectangles.

GUI:

Implement a simple GUI with visualization. Users can generate instances with random rectangles and chosen parameters: number of rectangles, min/max side lengths, and box length L . The GUI must allow repeated instance generation and applying both algorithms with different neighborhoods or selection strategies to the same instance.

Rectangular placements should be visualized between iterations such that the placement is clear, large enough, and informative. Transitions should not be confusing; automatically skip steps where changes are too small to observe.

Visualization Guidelines:

- Avoid overlapping information.
- Important elements: larger, thicker, strong colors.
- Unimportant elements: light neutral gray.
- Draw important elements last.
- Use strong contrasts for distinct elements; small contrasts for gradients.
- Prefer color-blind-friendly palettes (e.g., yellow/blue).
- Make small details easy to recognize.
- Assume viewers are unfamiliar with your GUI and need clarity.

Test Environment:

Create a test environment separate from the GUI, parameterized by tuples:

(number of instances, number of rectangles, two min side lengths, two max side lengths, box length)

For each tuple:

- generate the specified number of instances,
- run each algorithm on each instance,
- record objective values and CPU thread times.

Your test environment must support:

1. Small quick tests for demonstration.
2. Large exhaustive tests for correctness. Bring a protocol of such a run to the exam.

Tuning:

Since the pure algorithms are slow, you must optimize them. You may deviate from the pure form if necessary—for example by searching only promising parts of the neighborhood or tuning neighborhoods. If unsure, ask the instructor.

Performance Requirement:

Your implementation must satisfy:

For instances with up to 1000 rectangles, each algorithm must produce a solution within 10 seconds that visually cannot be improved further.